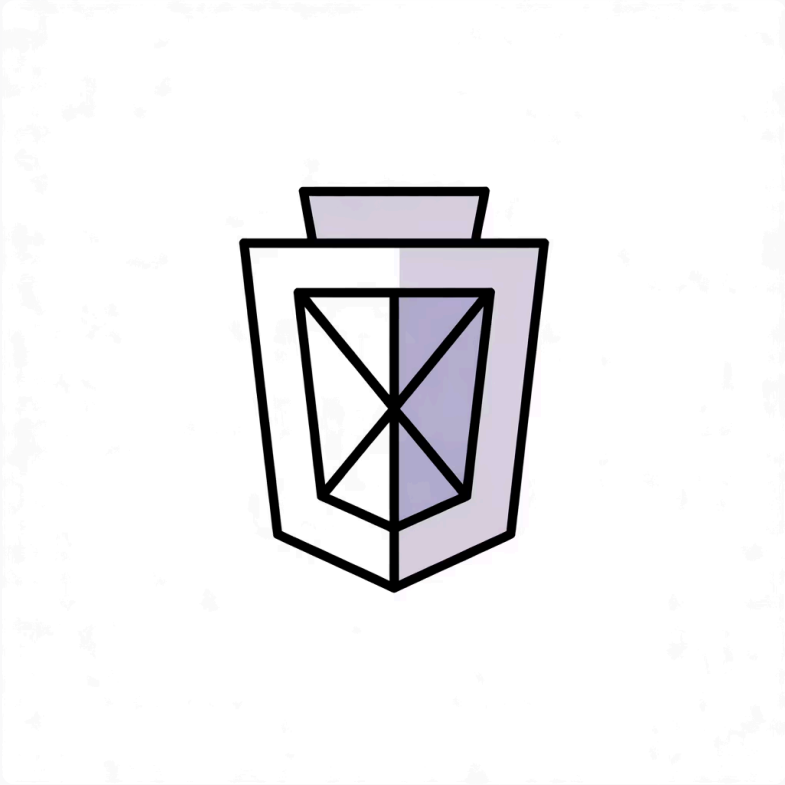
JavaScript - Jour 6

JSON, HTTP et Fetch API

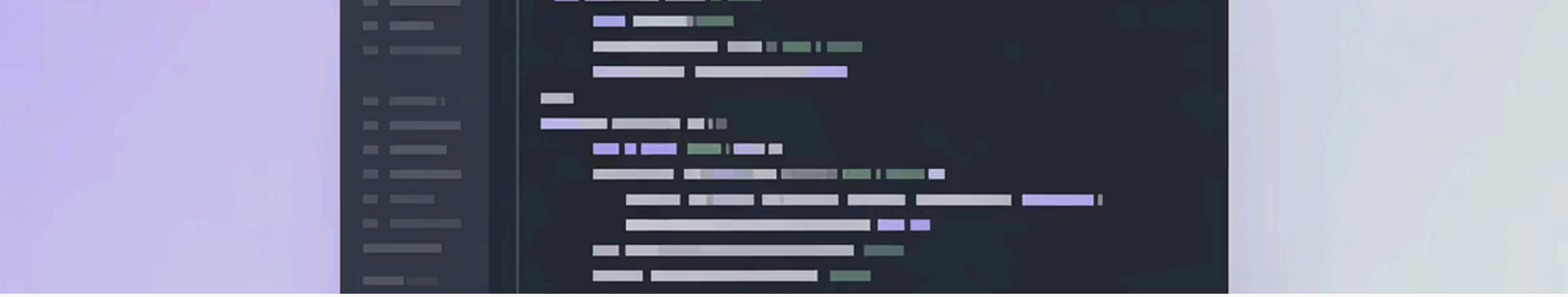


Objectifs pédagogiques de la journée

- Mise en place d'une API minimaliste
- Le format JSON et les requêtes GET
- Requêtes POST, PUT et DELETE
- Challenge de fin de journée

Avant de continuer…

On va s’installer un outils essentiel.



NodeJS, le runtime backend JavaScript

Même si notre objectif n'est pas de réaliser une application NodeJS, nous allons l'installer pour pouvoir gérer des dépendances utiles au développement.

Étapes

1

Se rendre sur le site officiel de NodeJS

nodejs.org

Node.js — Télécharger Node.js®

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line...

2

Sélectionner la version LTS (v24.13.0) et l'OS concerné (Windows, Linux ou Mac)

Cas de Windows : Téléchargez directement l'installateur (.msi)

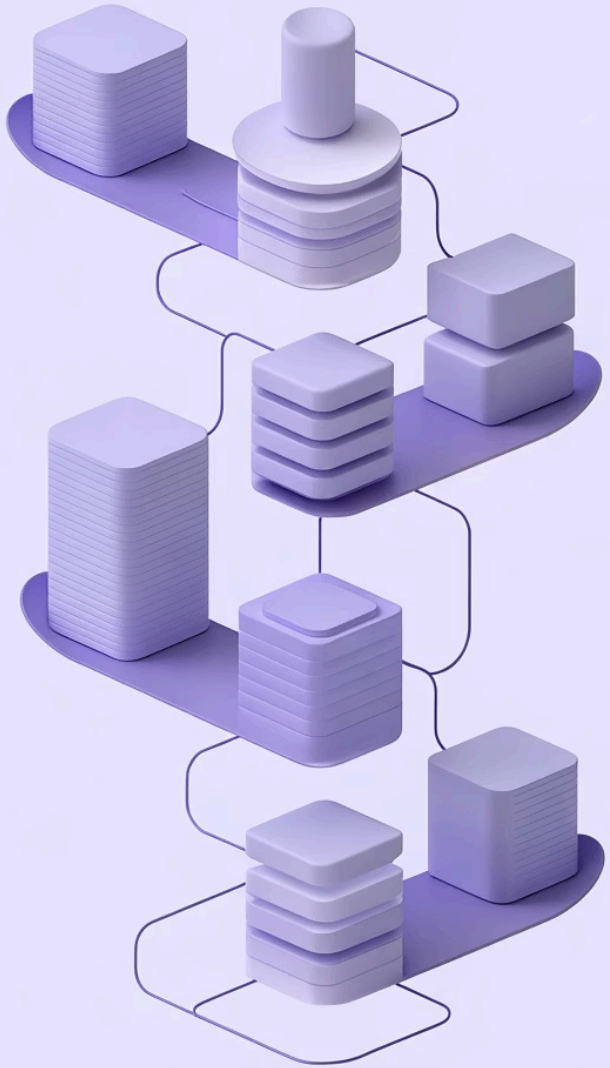
Cas de Linux/Mac/WSL : On utilise nvm pour pouvoir passer facilement d'une version à une autre.

3

Copier puis exécuter les commandes affichées

Pour Windows (sans WSL), suivez les instructions de l'installateur.

Pour les autres (y compris WSL), exécutez les commandes dans un terminal bash/zsh.



MODULE

Mise en place d'une API minimaliste

Installation guidée

Pour simuler une API REST locale rapidement, nous allons utiliser un outil très pratique : `json-server`**.**

Création d'un répertoire pour l'API

```
mkdir nodejs-json-server
cd nodejs-json-server
code .
```

~~Installation globale de json-server~~

```
npm install -g json-server
```

Installation guidée

Création du fichier de données (db.json)

`json-server` utilise un simple fichier JSON comme base de données. Créez un fichier nommé `db.json` à la racine de votre dossier de travail (le dossier qui contiendra votre API simulée, distinct de votre projet frontend).

▼ Contenu du fichier db.json

```
{
  "heroes": [
    {
      "id": 1,
      "name": "Aragorn",
      "class": "Guerrier",
      "level": 15,
      "hp": 120,
      "attack": 45,
      "defense": 30,
      "gold": 850
    },
    {
      "id": 2,
      "name": "Gandalf",
      "class": "Mage",
      "level": 20,
      "hp": 80,
      "attack": 65,
      "defense": 15,
      "gold": 1200
    },
    {
      "id": 3,
      "name": "Legolas",
      "class": "Archer",
      "level": 12,
      "hp": 90,
      "attack": 50,
      "defense": 20,
      "gold": 600
    }
  ],
  "quests": [
    {
      "id": 1,
      "title": "Sauver le village",
      "description": "Des gobelins attaquent le village voisin",
      "reward": 500,
      "difficulty": "Facile",
      "completed": false
    },
    {
      "id": 2,
      "title": "Le trésor du dragon",
      "description": "Récupérer le trésor gardé par un dragon",
      "reward": 2000,
      "difficulty": "Difficile",
      "completed": false
    }
  ],
  "inventory": [
    {
      "id": 1,
      "heroId": 1,
      "item": "Épée en acier",
      "type": "Arme",
      "bonus": 10
    },
    {
      "id": 2,
      "heroId": 2,
      "item": "Bâton magique",
      "type": "Arme",
      "bonus": 15
    }
  ]
}
```

Chaque clé de niveau supérieur (`heroes`, `quests`, `inventory`) représentera une ressource de votre API et sera accessible via une route spécifique.

Lancer le serveur pour l'API

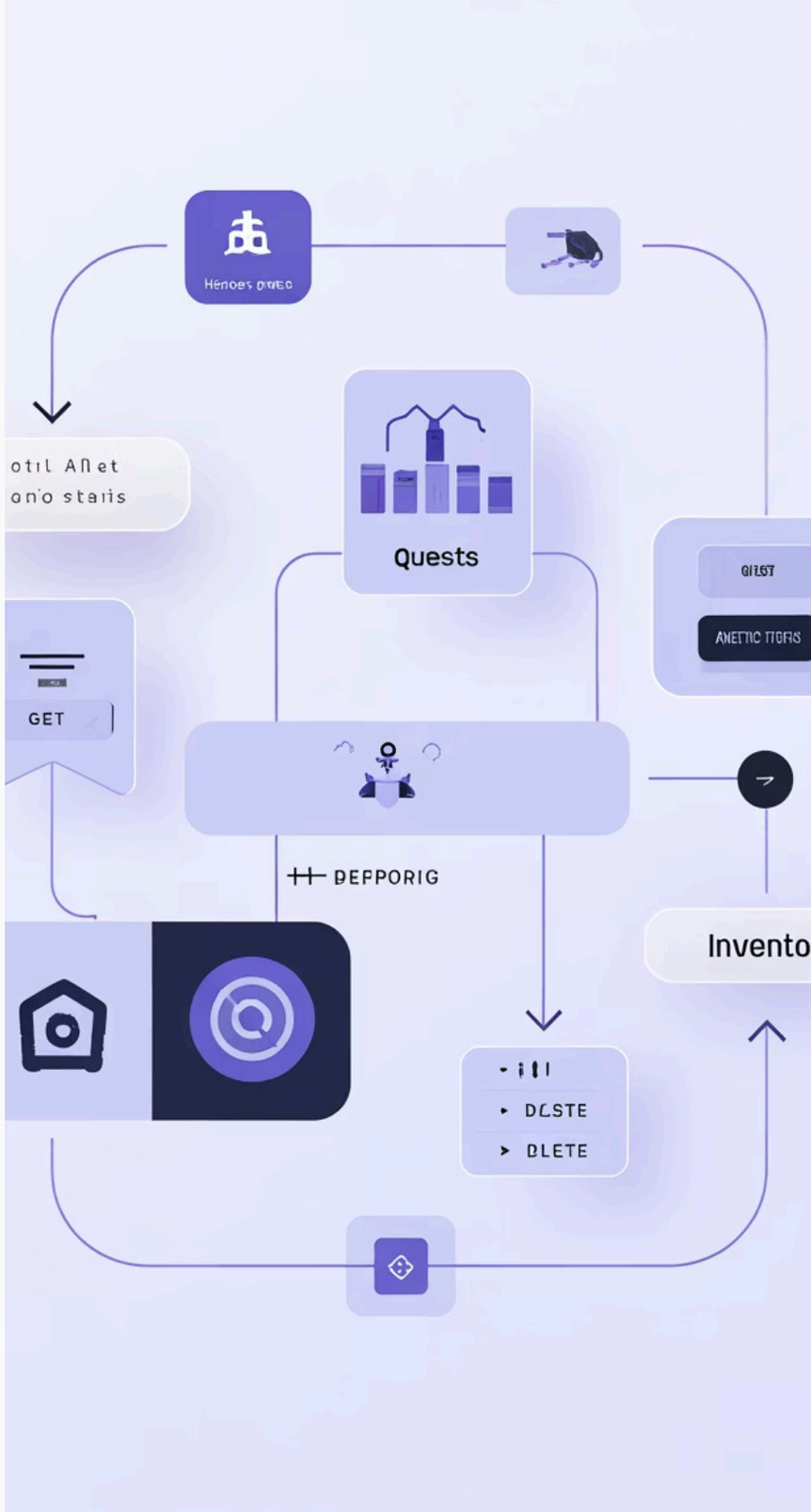
```
json-server --watch db.json --port 3000
```

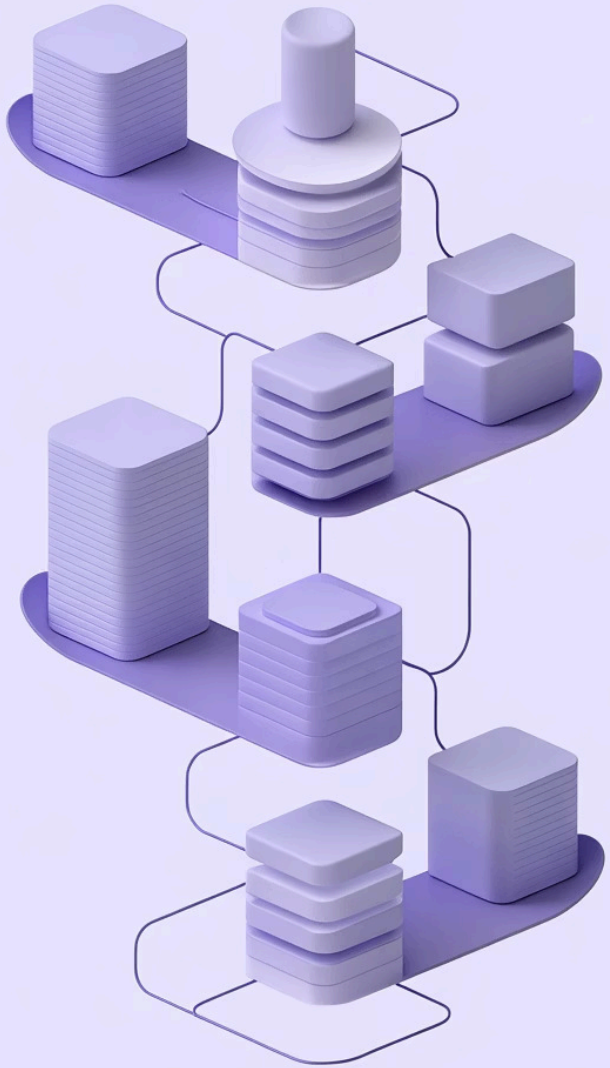
Le serveur sera accessible sur `http://localhost:3000`

Endpoints de l'API

Une fois votre serveur `json-server` lancé, plusieurs endpoints sont automatiquement créés et disponibles, basés sur les clés de votre fichier `db.json` :

Méthode	Endpoint	Description
GET	/heroes	Récupère la liste de tous les héros.
GET	/heroes/:id	Récupère un héros spécifique par son ID.
POST	/heroes	Crée un nouveau héros dans la base de données.
PUT	/heroes/:id	Met à jour un héros existant (remplacement complet).
PATCH	/heroes/:id	Met à jour partiellement un héros existant.
DELETE	/heroes/:id	Supprime un héros spécifique de la base de données.





MODULE

Le format JSON et les requêtes GET

Qu'est-ce que JSON ?

JSON (JavaScript Object Notation) est un format léger d'échange de données. Il est conçu pour être facile à lire et à écrire pour les humains, et facile à manipuler avec des scripts.

Syntaxe essentielle

- Les données sont structurées en paires **clé: valeur**.
- Les clés doivent impérativement être des **chaînes de caractères** (entourées de guillemets doubles).
- Les valeurs peuvent être des chaînes, des nombres, des booléens (`true`/`false`), des objets, des tableaux ou `null`.
- Les commentaires ne sont **pas autorisés** dans un fichier JSON.

```
{
  "name": "Aragorn",
  "level": 15,
  "is_hero": true,
  "items": ["épée", "armure"],
  "quest_status": null
}
```

Découverte de fetch

Pour interagir avec notre API (et toute API web), nous allons utiliser l'API `fetch` intégrée aux navigateurs modernes.

Elle fonctionne de manière asynchrone et retourne des Promesses, ce qui rend son utilisation fluide et non bloquante.

```
fetch("http://localhost:3000/heroes")
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error('Erreur :', error));
```

Anatomie d'une requête fetch

1. L'appel fetch(URL)

Initie la requête HTTP vers l'URL spécifiée. Il retourne une Promesse qui représente la réponse du réseau.

2. Traitement de la réponse (.then)

La première méthode `.then()` reçoit l'objet `Response`. On y vérifie souvent si la requête a réussi (`response.ok`) et on convertit le corps de la réponse en JSON via `response.json()`, ce qui retourne une autre Promesse.

3. Utilisation des données (.then)

La seconde méthode `.then()` est appelée une fois que les données JSON sont entièrement parsées. Elle reçoit les données brutes, prêtes à être utilisées dans votre application.

4. Gestion des erreurs (.catch)

La méthode `.catch()` est exécutée si une erreur survient à n'importe quelle étape de la chaîne des promesses (problème de réseau, URL incorrecte, erreur de parsing JSON, etc.).

Mise en pratique guidée

Pour mieux comprendre l'API `fetch`, nous allons créer une fonction asynchrone qui gérera la récupération et le traitement des données de nos héros via l'API.

1. Déclarer une fonction `async`

Encapsulez la logique dans une fonction fléchée asynchrone, permettant l'utilisation de `await` pour des opérations non bloquantes.

2. Récupérer les données avec `fetch`

Utilisez `await fetch(...)` avec l'URL de votre endpoint `/heroes` pour envoyer la requête HTTP.

3. Gérer la réponse et le JSON

Vérifiez si la réponse est valide (`response.ok`), puis parsez le corps de la réponse en JSON avec `await response.json()`.

4. Traiter et afficher les héros

Une fois les données reçues, parcourez-les et affichez les informations pertinentes dans la console.

5. Gérer les erreurs

Enveloppez toute l'opération dans un bloc `try...catch` pour intercepter et signaler toute erreur potentielle.

Mise en pratique guidée

Code final

```
const fetchHeroes = async () => {
  try {
    const response = await fetch("http://localhost:3000/heroes");

    if (!response.ok) {
      throw new Error(`Erreur HTTP : ${response.status}`);
    }

    const heroes = await response.json();

    console.log("Liste des héros :", heroes);

    heroes.forEach(hero => {
      console.log(`${hero.name} - Niveau ${hero.level} - ${hero.class}`);
    });

  } catch (error) {
    console.error("Erreur lors de la récupération :", error.message);
  }
};
```

Mise en pratique guidée

Votre défi : adapter la fonction existante pour renvoyer les données des héros et les afficher dynamiquement dans une page web.

1

1. Adapter la fonction `fetchHeroes`

Modifiez la fonction `fetchHeroes` pour qu'au lieu de simplement logger les héros, elle retourne le tableau de héros récupéré. Assurez-vous que le bloc `try...catch` gère toujours les erreurs.

2

2. Créer une fonction `renderHeroesTable`

Développez une nouvelle fonction, `renderHeroesTable`, qui prendra en argument une liste de héros. Cette fonction devra construire et retourner une chaîne de caractères HTML représentant un tableau.

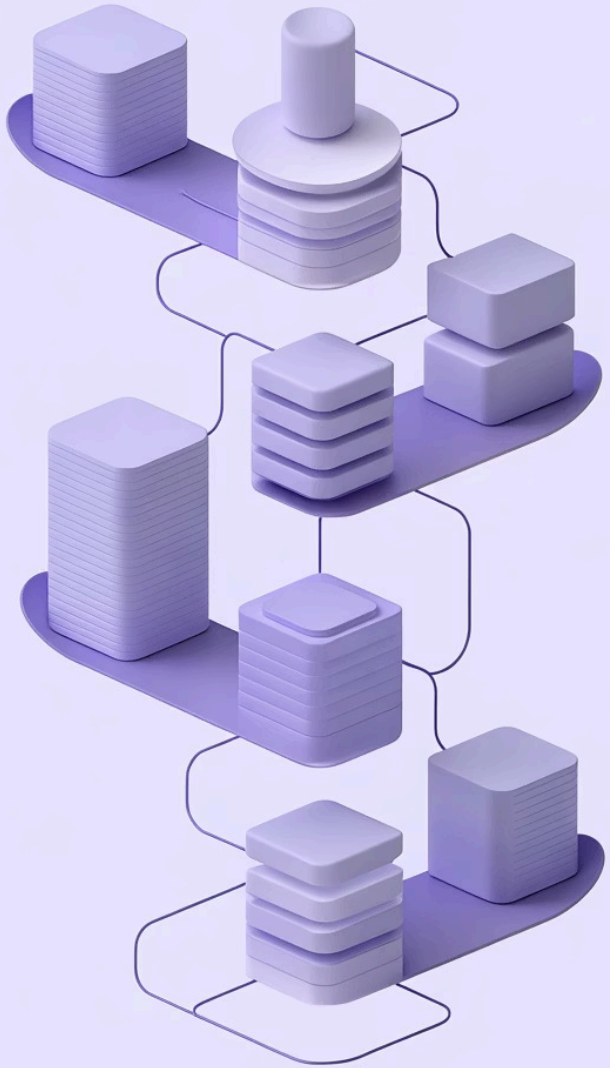
Ce tableau devra inclure les informations clés de chaque héros (nom, niveau, classe, ...).

3

3. Intégrer et afficher le tableau

Appelez `fetchHeroes` et, une fois les données récupérées, passez-les à votre fonction `renderHeroesTable`.

Insérez ensuite le HTML généré dans un élément de votre page (par exemple, un `<div id="app"></div>`).



MODULE

Requêtes POST, PUT et DELETE

Les méthodes HTTP POST, PUT et DELETE

POST : Créer une ressource

Envoie des données au serveur pour créer une nouvelle ressource. L'ID est généralement généré par le serveur en réponse à la requête.

PUT : Remplacer une ressource

Met à jour une ressource existante en la remplaçant intégralement avec les nouvelles données fournies. Nécessite l'ID de la ressource.

DELETE : Supprimer une ressource

Supprime une ressource spécifique du serveur en utilisant son identifiant unique. Cette action est irréversible.

PUT vs. PATCH : Quelle est la différence ?

PUT : Remplacement complet

Avec `PUT`, vous remplacez l'intégralité de la ressource cible par l'état décrit dans la requête. Si un champ de la ressource existante n'est pas inclus dans votre requête `PUT`, il sera **supprimé** ou réinitialisé.

PATCH : Mise à jour partielle

La méthode `PATCH` est utilisée pour appliquer des modifications **partielles** à une ressource. Seuls les champs que vous spécifiez dans la requête sont modifiés, les autres champs de la ressource restent inchangés.

Configuration des requêtes avec fetch

Pour envoyer des données ou effectuer des actions spécifiques (créer, modifier, supprimer) via l'API `fetch`, il est nécessaire de configurer la requête en ajoutant un objet d'options. Cet objet permet de contrôler la méthode HTTP, les en-têtes et le corps de la requête.

Méthode (`method`)

Indique le verbe HTTP de la requête (`'POST'`, `'PUT'`, `'DELETE'`). Si non spécifié, `fetch` utilise par défaut `'GET'`.

En-têtes (`headers`)

Un objet contenant les en-têtes HTTP de la requête, par exemple `'Content-Type': 'application/json'` pour indiquer le format des données envoyées.

Corps (`body`)

Les données à envoyer avec la requête. Pour les méthodes `POST` ou `PUT`, il doit souvent être une chaîne JSON, obtenue avec `JSON.stringify()`.

Exemple de configuration de requête avec fetch (POST)

Voici un exemple concret de la manière de configurer une requête `POST` avec `fetch` pour créer une nouvelle ressource sur le serveur. Nous envoyons un objet JavaScript qui sera converti en JSON dans le corps de la requête.

```
fetch("http://monapi.fr/maRessource", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(nouvelleRessource), // Convertit l'objet JS en chaîne JSON
});
```

Les deux méthodes essentielles de JSON

Pour manipuler des données au format JSON en JavaScript, deux méthodes sont à votre disposition :

`JSON.stringify()`

Cette méthode convertit un objet ou une valeur JavaScript (tableau, chaîne, nombre, booléen) en une chaîne de caractères JSON. C'est essentiel lorsque vous voulez envoyer des données JavaScript vers un serveur web via une API, car les serveurs attendent souvent des données sous forme de texte JSON.

`JSON.parse()`

Inversement, cette méthode analyse une chaîne de caractères JSON et la transforme en un objet ou une valeur JavaScript. Elle est utilisée pour convertir les réponses JSON reçues d'une API web en un format que votre application JavaScript peut manipuler directement.

Mise en pratique : Ajouter un nouvel héros

À l'aide des exemples précédents sur `fetch` et les requêtes POST, mettez en œuvre une fonction asynchrone capable d'envoyer les données d'un nouveau héros à votre API.

1. Intercepter la soumission du formulaire

Commencez par créer un formulaire HTML pour saisir les informations du héros (nom, niveau, classe). Ensuite, en JavaScript, interceptez l'événement `submit` de ce formulaire et utilisez `event.preventDefault()` pour empêcher le comportement par défaut du navigateur.

2. Préparer les données du héros

Récupérez les valeurs des champs du formulaire pour construire un objet JavaScript représentant votre nouveau héros. Cet objet devra ensuite être converti en une chaîne JSON à l'aide de `JSON.stringify()` pour le corps de la requête.

3. Envoyer la requête POST à l'API

Utilisez la fonction `fetch` pour envoyer une requête POST à l'endpoint `/heroes` de votre API. N'oubliez pas de configurer l'objet d'options avec la méthode (`'POST'`), les en-têtes (`'Content-Type': 'application/json'`) et le corps de la requête (votre chaîne JSON).

4. Gérer la réponse et les erreurs

Une fois la requête envoyée, vérifiez si la réponse est valide (`response.ok`). En cas de succès, vous pouvez, par exemple, afficher un message de confirmation ou actualiser la liste des héros. Assurez-vous d'implémenter un bloc `try...catch` pour gérer les erreurs potentielles.

Mise en pratique : Supprimer un héros

Maintenant que vous savez créer des ressources, continuez avec la suppression. Créez une fonctionnalité qui permet de supprimer un héros de votre liste via une requête DELETE à l'API.

1. Ajouter un bouton de suppression

Modifiez votre fonction `renderHeroesTable` pour inclure un bouton ou une icône "Supprimer" à côté de chaque héros, en lui ajoutant un attribut `data-id` avec l'ID du héros.

2. Écouter l'événement "click"

Attachez un écouteur d'événements à ces boutons. Lors d'un clic, récupérez l'ID du héros à supprimer via son attribut `data-id`.

3. Envoyer la requête DELETE

Utilisez `fetch` pour envoyer une requête DELETE à l'endpoint de l'API correspondant au héros (par ex. `/heroes/:id`). Spécifiez `method: 'DELETE'` dans l'objet de configuration.

4. Actualiser l'affichage

Après une suppression réussie, actualisez la liste des héros affichée dans votre tableau. Vous pouvez soit re-fetch toutes les données, soit supprimer la ligne du héros localement.

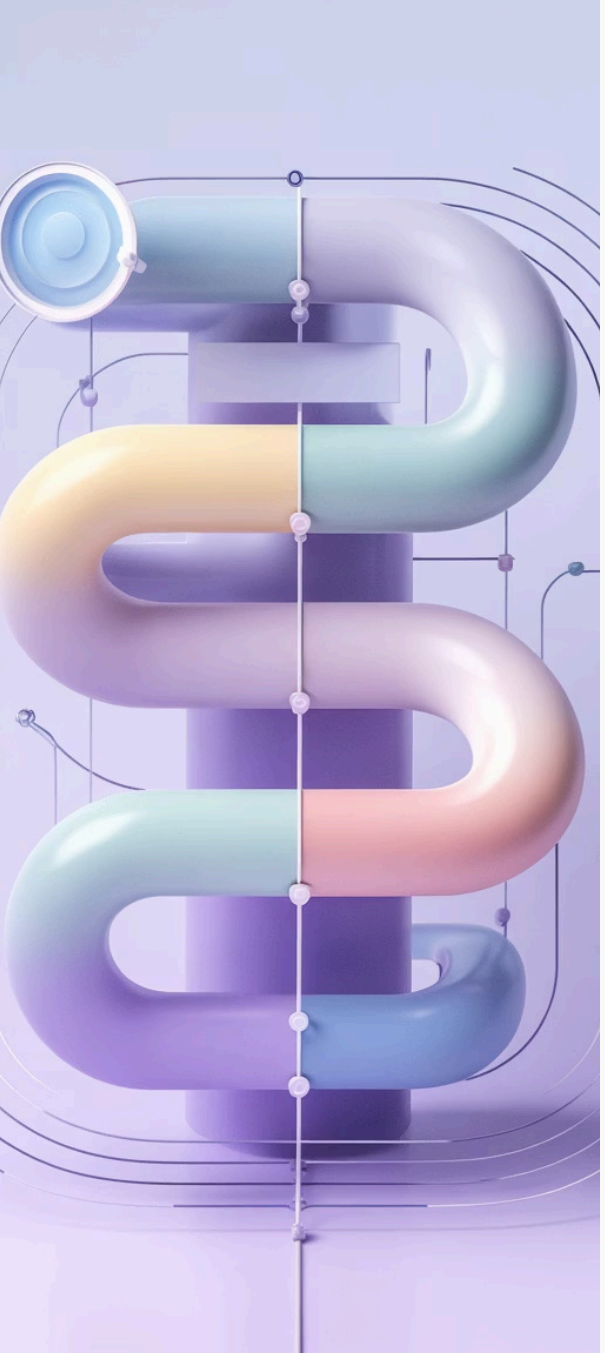
Bonus : Suppression multiple
Pour aller plus loin, intégrez des cases à cocher dans votre tableau. Permettez aux utilisateurs de sélectionner plusieurs héros et de les supprimer en parallèle (par exemple, avec `Promise.all`).

CONCLUSION

Et… on s'arrête là pour les notions du jour !

Ça fait beaucoup (vraiment), donc on fait une pause pour aujourd'hui.

… mais ce n'est pas tout à fait fini 😏



À SUIVRE

Prochaines étapes

Week-end bien méritez, mais ne pensez pas qu'on en a fini avec JavaScript 😉

On se retrouve lundi pour reprendre les bases de la POO et découvrir ce que permet JavaScript sur le sujet (car ce n'est pas exactement pareil qu'avec PHP que vous connaissez déjà).

Et comme d'habitude, ce qu'on a vu jusqu'à aujourd'hui sera approfondi au fur et à mesure !

PROJET DE FIN DE JOURNÉE

Projet de fin de journée : Votre Pokédex

Préparez-vous à mettre en pratique toutes vos connaissances sur la consommation d'API avec `fetch` en construisant une application réelle !

Votre mission est de créer une interface graphique interactive pour un Pokédex, en utilisant la célèbre **PokéAPI** qui est publique et regorge de données.

Votre Pokédex devra afficher :

Numéro du Pokémon

Affichez l'identifiant unique de chaque Pokémon (son ID).

Nom du Pokémon

Affichez le nom français du Pokémon (nécessite une requête supplémentaire vers les espèces de Pokémon).

Sprite de face

Montrez le sprite frontal du Pokémon (normal ou shiny).

Type(s) du Pokémon

Indiquez le ou les deux types élémentaires de chaque Pokémon.

Bonus 1 : Au clic sur un Pokémon, ouvrez une fenêtre modale affichant ses statistiques détaillées.

Bonus 2 : Toujours au clic sur un Pokémon, jouez son cri emblématique à l'ouverture de la fenêtre modale.

Bonus 3 : Permettez d'intervertir entre la vue des sprites normaux et shiny avec une checkbox

Bonus 4 : Trop facile tout ça ? Alors remplacez l'utilisation de l'API Rest par l'API GraphQL de PokéAPI 💪